



VYTAL BACKEND

Implementation & Testing Guide

Secure dynamic patient records, atomic clinical workflows, account billing, donations, receipts and automated policy tests.

TASK 01	Dynamic database	COMPLETE
TASK 02	Scoreboard	DEFERRED
TASK 03	Account billing	COMPLETE
TASK 04	Donations + receipts	COMPLETE

Branch: agent/secure-database-billing | Prepared: 19 August 2026



01 / DELIVERY ARCHITECTURE

What the backend now guarantees

The browser is treated as untrusted. Ownership, clinical validation, payment state and pricing are enforced at server and database boundaries.

BROWSER	EDGE / RPC	POSTGRES / PROVIDER
Authenticated React client Owner-scoped reads No card data or service keys	Origin + JWT checks Rate limits Transactional clinical functions Stripe session creation	Forced RLS Private audit events Signed Stripe webhook Server-owned product prices

Clinical write path

- `save_account_profile(...)` validates and commits the profile plus all account baselines atomically.
- `record_screening(...)` verifies account ownership and active patient consent before accepting observations.
- The database derives risk status, creates observations, opens a referral when needed, and emits a private audit event in one transaction.
- `advance_referral(...)` allows only forward care stages or cancellation; browser clients cannot skip directly to completed.

Payment write path

- The browser sends a product code, never a trusted price. The Edge Function reads the active amount from `billing_products`.
- Stripe-hosted Checkout owns card entry. VYTAL stores only IDs, amount, currency, status and receipt link.
- Only a correctly signed webhook can set paid status, after matching checkout ID, owner ID, amount and currency.
- Payment-event claims make provider retries idempotent and recover stale/failed processing safely.

This is a strong prototype security boundary, not a claim of medical-device certification or regulatory compliance.



02 / TASK 01

Dynamic database and clinical workflow test

Run these tests with test identities and synthetic screening values only.

Application test

- 1. Sign in as Worker A. Save name, phone, emergency contact and a resting heart-rate baseline; refresh and verify persistence.
- 2. Register Patient A with consent granted. Register Patient B under a second worker account.
- 3. Run a normal screening for Patient A. Verify one screening plus its metric observations and no referral.
- 4. Run a flagged screening. Verify one referral appears with a database-derived reason and priority.
- 5. Advance flagged -> referred -> contacted -> appointment booked -> completed. Verify stage skipping is rejected.
- 6. As Worker A, try to read or write Patient B. Verify the database returns no row or rejects the request.
- 7. Withdraw consent and try another screening. Verify the database rejects it.

Database policy test

```
supabase start
supabase test db
```

Expected coverage

CHECK	EXPECTED
Owner isolation	Each worker sees only owned patients, screenings, referrals and baselines.
Atomicity	Screening, observations, referral and audit event either all commit or all roll back.
Write boundary	Direct browser inserts/updates to protected clinical rows are denied.
Referral audit	Creation and transitions append events; browser clients cannot forge them.

PASS means the pgTAP plan completes with zero failed assertions.



03 / TASKS 03-04

Billing, donation and receipt test

Use Stripe test mode only. Do not enter a real card or switch to live keys during development.

Deploy prerequisites

```
supabase functions deploy ai-explanation
supabase functions deploy create-billing-checkout
supabase functions deploy create-donation-checkout
supabase functions deploy stripe-webhook --no-verify-jwt
```

Stripe webhook events

- checkout.session.completed
- checkout.session.async_payment_succeeded
- checkout.session.async_payment_failed
- checkout.session.expired

End-to-end cases

CASE	ACTION	EXPECTED RESULT
Invoice success	Buy the server-listed service with 4242 4242 4242 4242.	Invoice becomes paid after webhook; receipt link appears.
Donation success	Donate a test amount through hosted checkout.	Donation becomes paid after webhook; receipt link appears.
Cancel	Exit checkout without payment.	No paid state. Stripe expiry moves pending row to void/cancelled.
Duplicate event	Resend the same Stripe event.	Second delivery is acknowledged but not processed twice.
Invalid signature	Call webhook without a valid Stripe signature.	HTTP 400; no payment row changes.
Mismatch	Replay a signed test fixture with mismatched stored amount/currency.	Processing fails; invoice/donation never becomes paid.
Rate limit	Start more than five checkouts within ten minutes.	HTTP 429 for attempts above the account limit.

Never mark an invoice or donation paid from a success-page redirect. The signed webhook is authoritative.



04 / WINDOWS WORKFLOW

Run, verify and publish from VS Code

Use the integrated PowerShell terminal while it is opened at the repository root.

Install and run

```
cd C:\Users\A\OneDrive\Documents\GitHub\Vital
npm.cmd install
npm.cmd run verify
npm.cmd run dev
```

Why npm.cmd?

PowerShell may block npm.ps1 under the current execution policy. npm.cmd runs the same npm installation without changing the machine-wide policy.

Expected verification

- Security scan reports no browser secrets and no localStorage patient persistence.
- Backend contract check confirms protected RPC, price, signature and idempotency boundaries.
- Vite production build completes. A bundle-size warning is non-blocking but should be optimized later.
- GitHub Actions additionally runs the real pgTAP database policy suite with local Supabase.

Import and publish the prepared bundle

```
$bundle = "$env:USERPROFILE\Downloads\VYTAL_Backend_All_Tasks_Final.bundle"
git switch agent/secure-database-billing
git fetch $bundle agent/secure-database-billing
git merge --ff-only FETCH_HEAD
git push -u origin agent/secure-database-billing
```

Do not run git add . when the working tree contains unrelated personal files. Stage only the intended backend, UI, documentation and workflow files.



05 / DEPLOYMENT

Safe activation checklist

Complete the checklist in test mode before using any production service.

- [] Apply migrations 001, 002, 003 and 004 in filename order.
- [] Set only VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY in the browser environment.
- [] Set Stripe, webhook, AI, site URL and allowed-origin values only as Edge Function secrets.
- [] Verify RLS is enabled and forced on public patient, clinical and billing tables.
- [] Deploy all four Edge Functions and configure the four Stripe Checkout webhook events.
- [] Run npm.cmd run verify and supabase test db.
- [] Complete successful, cancelled, delayed, duplicate, invalid-signature and mismatch payment tests.
- [] Turn on MFA for GitHub, Supabase, Stripe, hosting and AI-provider administrators.
- [] Enable backups and perform a restore drill; define retention and deletion rules.
- [] Add monitoring for webhook failures, repeated auth failures, rate-limit events and RLS denials.

Production gate

Before handling real patient information or live payments, obtain a professional penetration test, privacy/compliance review, clinical validation, incident-response plan, merchant/legal approval, and written ownership of operational responsibilities.

Evidence in the repository

AREA	FILES
Schema	supabase/migrations/202608190001...004
Database tests	supabase/tests/database/001_clinical_backend.sql; 002_billing_backend.sql
Edge Functions	ai-explanation; create-billing-checkout; create-donation-checkout; stripe-webhook
Static checks	scripts/security-check.mjs; scripts/backend-contract-check.mjs
CI	.github/workflows/verify.yml
Operations	SECURITY.md; docs/BACKEND_IMPLEMENTATION.md